

03/09/2026

* Python

→ 1. lists are

: python ↓

1. ordered
2. mutable
3. allow duplicates

```
nums = [10, 20, 30, 40]
nums[0]
nums[-1]
nums.append(50)
nums.remove(20)
num.pop()
```

List comprehension =

instead of : squares = []

for x in range(10):

squares.append(x**2)

we can write : squares = [x**2 for x in range(10)]

with condition : evens = [x for x in range(20) if x%2==0]

→ 2. tuples are

: python ↓

1. ordered
2. immutable
3. allow duplicates

```
point = (10, 20)
```

⇒ useful for values shouldn't change.

example : x, y = point

this is called unpacking — extract elements from

print(x) #10

print(y) #20

to individual var.

→ 3. Sets are =

1. duplicate removal
2. fast membership checking
↓
average lookup
 $O(1)$

python ↴

```
nums = {1, 2, 3, 4, 3}
print(nums)
```

example :

```
seen = set()
for x in [1, 2, 2, 3]:
    if x in seen:
        print("duplicate", x)
    seen.add(x)
```

→ 4. Dictionaries are one of most imp structures.

→ student = { "name": "Nv",
 "score": 90 }

→ access:

student["name"]

→ safe access:

student.get("age", 0)

→ add:

student["age"] = 21

→ iteration:

```
for key, value in student.items():
    print(key, value)
```

→ 5. Functions

→ def add(a, b):
 return a+b

→ default argument

def greet(name = "User"):
 return f"Hello {name}"

→ keyword argument

greet(name = "Necyati")

→ variable-length positional arguments

def total(*args):
 return sum(args)

→ variable-length keyword arguments

def info(**kwargs):
 print(kwargs)

→ 6. Lambda - a small anonymous function

→ normal:

def square(x):
 return x**2

→ lambda:

square = lambda x: x**2

} useful with
• sorted()
• map()
• filter()

→ 5. Functions

→ `def add(a, b):`
 `return a + b`

→ default argument

`def greet(name = "User"):`
 `return f "Hello {name}"`

→ keyword argument

`greet(name = "Necyati")`

→ variable-length positional arguments

`def total(*args):`
 `return sum(args)`

→ variable-length keyword arguments

`def info(**kwargs):`
 `print(kwargs)`

→ 6. Lambda - a small anonymous function

→ normal:

`def square(x):`
 `return x ** 2`

→ lambda:

`square = lambda x: x ** 2`

} useful with

- `sorted()`
- `map()`
- `filter()`

→ 7. map() and filter()

map - apply function to every element

→ `nums = [1, 2, 3, 4]`

`result = list(map(lambda x: x ** 2, nums))`

filter - keeps values satisfying a condition

→ `nums = [1, 2, 3, 4, 5]`

`evens = list(filter(lambda x: x % 2 == 0, nums))`

→ 8. Exception handling

try :

`x = int(input())`

`result = 10/x`

except ValueError :

`print("Invalid")`

except ZeroDivisionError :

`print("Cannot divide by zero")`

finally :

`print("Finished")`

try → code that might fail

except → handle error

finally → runs regardless

→ 9. OOP quick rev

class Model:

```
def __init__(self, name):
```

```
    self.name = name
```

```
def train(self):
```

```
    print(f"Training {self.name}")
```

→ create object:

```
model = Model("Random Forest")
```

```
model.train
```

→ Architecture:

class

├── attributes

└── methods

↓
obj

terms - class, object, constructor, abstraction

inheritance, encapsulation, polymorphism

→ 10. Inheritance

class Model:

```
def train(self):
```

```
    print("training")
```

class NeuralNetwork(Model):

```
def predict(self):
```

```
    print("predicting")
```

Now ↓

nn = NeuralNetwork()

nn.train()

nn.predict()

⇒ NeuralNetwork inherits from Model

→ 11. Files

• reading :

with open("data.txt", "r") as file:

data = file.read()

• writing :

with open("result.txt", "w") as file:

file.write("Hello")

Why with? - cause python automatically closes the file.